From Notebook to Django: the Broker Commission App

Nov 5, 2026 · @tanvik

Overview

The notebook becomes a web page: log in, upload the brokers-with-referrals CSV, pick a month and a rate, and download the styled two-sheet Excel. Every run is saved, with its numbers, so past reports can be downloaded again. The finished project is in `nexora_reports_django.zip`. All nine of its tests pass, and the screens were checked in a browser.

The main idea: **your pandas code moves almost unchanged into a plain Python file.** Django only adds the parts around it: a form, pages, a database table to remember runs, and file storage.

Where each part of the notebook went

Notebook	App file	What changed
`get_final_ledger(as_pandas=True)`	`ledger.py`	Reads only the report's month, clients and ledger types, through `scan_ledger()`
`pd.read_csv(r"...brokers...csv")`	`services.read_brokers()` + `forms.py`	The file comes from an upload; columns and size are checked
Month filter, `isin(child)`, BILL/NBP filter, `pnl`	`services.client_pnl()`	One function with `start`, `end`, `clients`, `rate`
`result = pd.merge(...)`	`services.client_sheet()`	Unchanged logic
`brokers = result.groupby(...)`	`services.broker_sheet()`	Unchanged logic

Notebook	App file	What changed
`style_sheet()` + `ExcelWriter(path)`	`excel.build_workbook()`	Written to memory (`BytesIO`), not a fixed path
Changing the month and path by hand	`forms.py` + `views.py`	Chosen on the page, checked, and saved per run

Two bugs fixed on the way

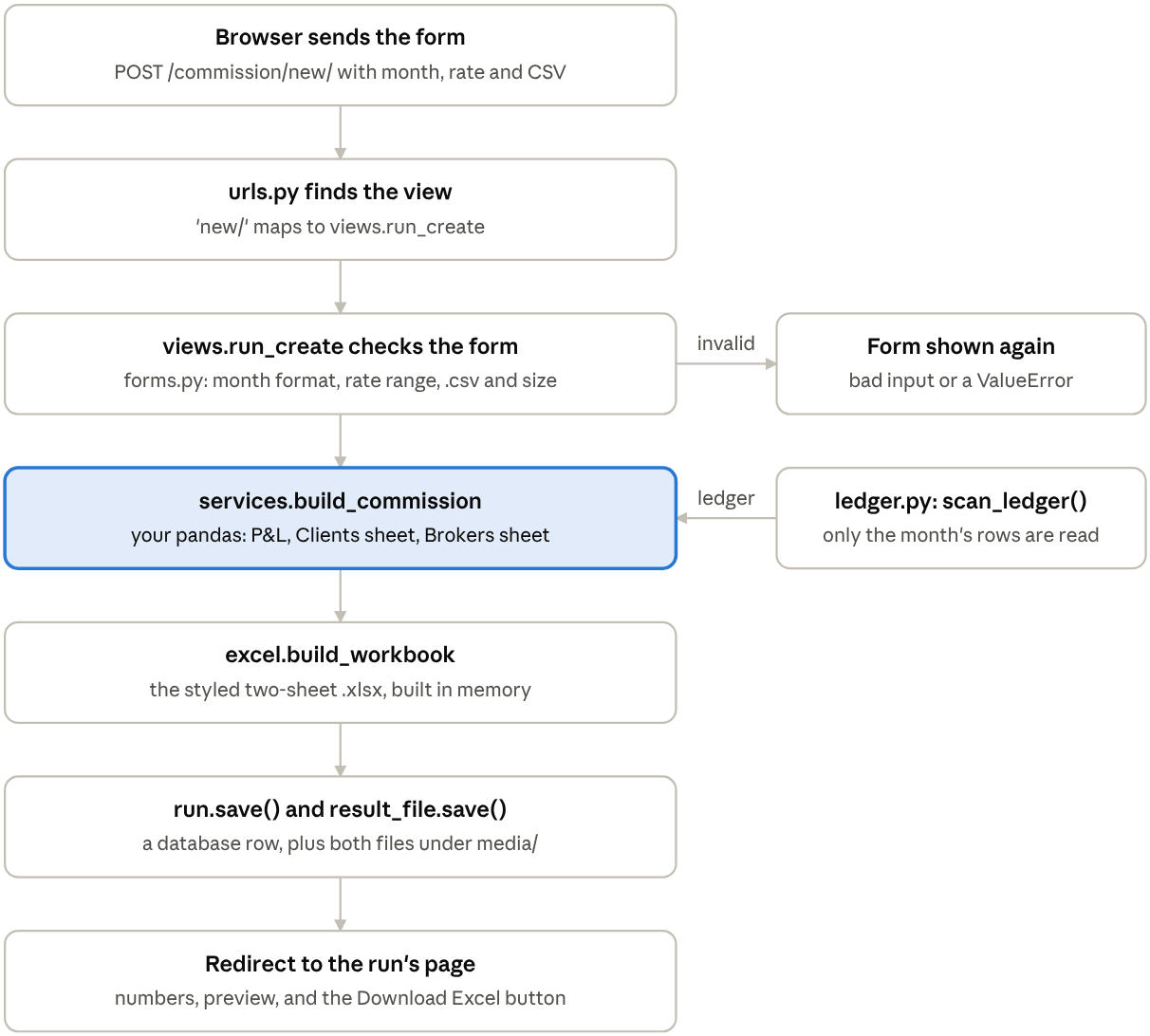
- **The month filter:** the notebook's `led_oct` variable held **August** (`'2026-08'`). The app takes the month from the form, so the name can't disagree with the data.
- **NBP is counted again:** the notebook filtered BILL rows before keeping BILL **and** NBP, so NBP was always 0. `client_pnl` filters both types in one step.

The files

```
nexora_reports/
    manage.py
    nexora_reports/          settings.py, urls.py          (the project)
    commission/              (the app)
        services.py          the pandas calculation, no Django code
        excel.py             the styled workbook
        ledger.py            reading only the rows a report needs
        demo_ledger.py       made-up data, so it runs before you connect
yours
        models.py            CommissionRun: one saved report
        forms.py             checking the inputs
        views.py             the four pages
        urls.py              page addresses
        admin.py             runs in the Django admin
        templates/           the HTML
        tests.py             nine tests
```

How one report request flows

One request passes through six files; only one holds the maths



one report request · 7 steps

Everything except the highlighted step is Django plumbing you write once. When a commission rule changes, only `services.py` changes.

Step 1: create the project and the app

A Django **project** is the whole website: settings, top-level URLs, the database connection. An **app** is one feature inside it. Here, `commission` is the only app; a dialer report could later be a second app in the same project.

```
pip install django pandas openpyxl
django-admin startproject nexora_reports
cd nexora_reports
python manage.py startapp commission
```

Settings changes (`nexora_reports/settings.py`)

```
INSTALLED_APPS = [
    # ...Django's own apps...
    'commission',                      # tell Django the app exists
]
TIME_ZONE = 'Asia/Dubai'

MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"        # where uploaded CSVs and generated
Excel files are stored

LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "commission:run_list"
LOGOUT_REDIRECT_URL = "login"

LEDGER_MODULE = os.environ.get("LEDGER_MODULE", "commission.demo_ledger")
LEDGER_MODULE_PATH = os.environ.get("LEDGER_MODULE_PATH", "")
LEDGER_BASE_PATH = os.environ.get("LEDGER_BASE_PATH", "")          # empty:
use the path inside ledger_loader.py
LEDGER_APPEND_FOLDER = os.environ.get("LEDGER_APPEND_FOLDER", "")
LEDGER_ONLY_NEW_APPENDS = os.environ.get("LEDGER_ONLY_NEW_APPENDS",
"false").lower() == "true"
LEDGER_CACHE_MINUTES = 30                                           # only
for loaders without scan_ledger()
```

- **`INSTALLED_APPS`**: without `'commission'` here, Django ignores its models and templates.
- **`MEDIA_ROOT`** is the folder for user files. Django keeps the files on disk and only their paths in the database.
- **`LEDGER_MODULE`** names the module that loads the ledger. Changing it from `commission.demo_ledger` to `ledger_loader` switches to your real data, with no code change.
- **`LEDGER_BASE_PATH`** and **`LEDGER_APPEND_FOLDER`** can stay empty, so the paths written inside `ledger_loader.py` are used. Set them only to point the app somewhere else.

- **`os.environ.get(..., default)`** lets you override a setting with an environment variable, the same idea as `UNO_API_TOKEN`.

Step 2: services.py, your notebook as functions

This is the most important file, and it has **no Django imports**. Each notebook step becomes a function that takes tables in and returns a table. You can test it on tiny frames, call it from a notebook, and change it without touching the web parts.

```
TRADING_TYPES = ["BILL", "NEGATIVE BALANCE PROTECTION"]
BROKER_COLUMNS = ["Broker Name", "Broker Client ID", "Broker Referral Count",
"Referred User Client ID"]

def month_range(month):
    """'2026-08' -> (2026-08-01, 2026-09-01): a half-open range."""
    start = pd.Period(month, freq="M").start_time
    return start, start + pd.offsets.MonthBegin(1)

def read_brokers(file):
    df = pd.read_csv(file, dtype=str)                # a path or an uploaded
file both work
    missing = set(BROKER_COLUMNS) - set(df.columns)
    if missing:
        raise ValueError(f"The broker file is missing columns: {',
'.join(sorted(missing))}")
    ...

def client_pnl(led, clients, start, end, rate=0.05):
    created = pd.to_datetime(led["CreatedAt"])
    ledger_type = led["LedgerType"].astype(str).str.strip().str.upper()
    keep = ((created >= start) & (created < end)
            & led["ClientID"].isin(set(clients)) &
ledger_type.isin(TRADING_TYPES))
    rows, lt = led[keep], ledger_type[keep]
    ...                                              # the same assign /
groupby / assign as the notebook

def client_sheet(brokers, pnl): ...                  # your result =
pd.merge(...)
def broker_sheet(clients, rate=0.05): ...            # your brokers =
result.groupby(...)
```

```
def build_commission(led, brokers, month, rate=0.05):
    start, end = month_range(month)
    pnl = client_pnl(led, brokers["Referred User Client ID"], start, end,
rate)
    clients = client_sheet(brokers, pnl)
    brokers_out = broker_sheet(clients, rate)
    return {"clients": clients, "brokers": brokers_out, "summary": {...}}
```

Why it's built this way

- **One job per function.** `build_commission` reads like the notebook's story, top to bottom, and each piece can be tested alone.
- **Errors as `ValueError` with a clear message.** The view catches them and shows the message on the page, instead of a crash.
- **Inputs are cleaned inside:** spaces stripped from IDs, ledger types upper-cased, blank amounts treated as 0, so a messy upload still works.
- **`rate` is a parameter** used in both the client commission and the broker brokerage, so the page's rate field drives both.

Step 3: excel.py, the workbook in memory

A web app shouldn't write to a fixed path like `Downloads\Broker_Commission_Aug.xlsx`: two people running reports at once would overwrite each other. `build_workbook` writes into a `BytesIO`, a file that lives in memory, and returns the bytes. The view then hands them to Django to store.

```
from io import BytesIO

def build_workbook(brokers, clients) -> bytes:
    buffer = BytesIO()                                      # an in-memory
file
    with pd.ExcelWriter(buffer, engine="openpyxl") as writer:
        brokers.to_excel(writer, sheet_name="Brokers", index=False,
startrow=1)
        clients.to_excel(writer, sheet_name="Clients", index=False,
startrow=1)
        style_sheet(writer.sheets["Brokers"], BROKER_BANDS, ...)
        style_sheet(writer.sheets["Clients"], CLIENT_BANDS, ...)
    return buffer.getvalue()                                # the finished
.xlsx as bytes
```

`style_sheet` is your function, unchanged. The band definitions moved to the top of the file as constants (`BROKER_BANDS`, `CLIENT_BANDS`), so changing a title or colour is one edit.

Step 4: ledger.py, read only what the report needs

Your `ledger_loader.py` builds the ledger from Polars **lazy scans**: a plan for reading `Ledger.csv` and the append files, where nothing is read until `.collect()`. The updated loader adds one function, `scan_ledger()`, which returns that plan instead of collecting it. `get_final_ledger()` now calls it and behaves exactly as before; this was checked with and without `only_new_appends`.

The app filters the plan by **month, clients and ledger type before collecting**, so each report reads only its own rows from the CSVs. The full ledger never sits in the server's memory.

```
def get_ledger_slice(start, end, clients=None, types=None):
    module = _module()                                      # ledger_loader,
from LEDGER_MODULE_PATH
    if hasattr(module, "scan_ledger"):
        return _slice_lazy(module, start, end, clients, types)
    return _slice_from_cache(module, start, end, clients, types)   # older
loaders: load once, cache

def _slice_lazy(module, start, end, clients, types):
    lf = module.scan_ledger(columns=COLUMNS,
only_new_appends=settings.LEDGER_ONLY_NEW_APPENDS)

    keep = (pl.col("CreatedAt") >= start) & (pl.col("CreatedAt") < end)
    if clients is not None:
        keep = keep &
pl.col("ClientID").cast(pl.String).str.strip_chars().is_in(ids)
    if types is not None:
        keep = keep &
pl.col("LedgerType").cast(pl.String).str.strip_chars().str.to_uppercase().is_
in(types)

    rows, bad = pl.collect_all([lf.filter(keep),
# the report's rows
                                lf.select(pl.col("CreatedAt").null_count())])
# unreadable dates
    ...
    return rows.to_pandas(), notes
```

The view calls it right after reading the broker CSV:

```
start, end = services.month_range(run.month)
led, notes = get_ledger_slice(start, end,
                              clients=brokers["Referred User Client ID"],
                              types=services.TRADING_TYPES)
report = services.build_commission(led, brokers, run.month, float(run.rate))
for note in notes:
    messages.warning(request, note)             # shown as an amber banner
on the next page
```

New ideas here

- **Filter, then collect.** The same rule as in your notebooks: a LazyFrame filtered before `.collect()` only loads the rows that pass. Tested on files shaped like yours, it returned exactly the rows a full load plus a pandas filter gives.
- **`pl.collect_all([...])`** runs two plans that share the same files in one pass: the report's rows, and the count of rows whose `CreatedAt` couldn't be read. Your loader's warning becomes a message on the page instead of disappearing into the server log.
- **`hasattr(module, "scan_ledger")`** keeps older loaders working. Without `scan_ledger`, the app loads the whole ledger once, keeps it for `LEDGER_CACHE_MINUTES`, and filters in pandas.
- **`importlib.import_module(settings.LEDGER_MODULE)`** turns the module name in settings into the module itself, so one settings line switches between the demo and your loader.
- **`only_new_appends`** comes from `LEDGER_ONLY_NEW_APPENDS`. It's off by default to match your notebooks; turn it on if append files overlap `Ledger.csv`, or those rows count twice.

Step 5: models.py, remembering each run

A model is a Python class that Django turns into a database table. `CommissionRun` stores one row per generated report: the inputs, the two files, and the headline numbers.

```
class CommissionRun(models.Model):
    month = models.CharField(max_length=7, help_text="YYYY-MM, e.g. 2026-08")
    rate = models.DecimalField(max_digits=5, decimal_places=4,
default=Decimal("0.05"))
    broker_file = models.FileField(upload_to="commission/inputs/%Y/%m/")
    result_file = models.FileField(upload_to="commission/results/%Y/%m/",
blank=True)

    brokers_count = models.PositiveIntegerField(default=0)
    clients_count = models.PositiveIntegerField(default=0)
    loss_brokers = models.PositiveIntegerField(default=0)
    total_brokerage = models.DecimalField(max_digits=14, decimal_places=2,
default=Decimal("0"))

    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True, on_delete=models.SET_NULL)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]                          # newest first
everywhere
```

Field type	Stored as	Why here
`DecimalField`	An exact decimal	Money and rates; floats can give 0.30000000000000004
`FileField`	The file's path; the file itself goes under `MEDIA_ROOT`	Uploaded CSV and generated Excel
`ForeignKey(User, on_delete=SET_NULL)`	The user's id	Who ran it; the run survives if the user is deleted
`auto_now_add=True`	Set once, when created	The run's timestamp

Create the table

```
python manage.py makemigrations commission     # writes
migrations/0001_initial.py from the class
python manage.py migrate                       # creates the table in the
database
```

A **migration** is a script describing a change to the database. Change the model later (say, add a `notes` field), run both commands again, and Django updates the table without losing data.

See runs in the admin

```
@admin.register(CommissionRun)
class CommissionRunAdmin(admin.ModelAdmin):
    list_display = ("month", "rate", "brokers_count", "clients_count",
"loss_brokers",
                    "total_brokerage", "created_by", "created_at")
    list_filter = ("month",)
```

After `python manage.py createsuperuser`, `/admin/` shows every run with a filter by month, for free.

Step 6: forms.py, checking inputs before any work

A `ModelForm` builds the HTML fields from the model and checks what the user sends. Each `clean_<field>` method validates one field; raising `ValidationError` shows your message next to that field and stops the run.

```
class CommissionRunForm(forms.ModelForm):
    class Meta:
        model = CommissionRun
        fields = ["month", "rate", "broker_file"]
        widgets = {"month": forms.TextInput(attrs={"type": "month"})}     #
the browser's month picker

    def clean_month(self):
        month = self.cleaned_data["month"]
        if not re.fullmatch(r"\d{4}-(0[1-9]|1[0-2])", month):
            raise forms.ValidationError("Use the format YYYY-MM, e.g. 2026-
08.")
```

```
        return month

    def clean_rate(self):
        rate = self.cleaned_data["rate"]
        if not Decimal("0") < rate <= Decimal("1"):
            raise forms.ValidationError("The rate must be above 0 and at most
1 (0.05 means 5%).")
        return rate

    def clean_broker_file(self):
        f = self.cleaned_data["broker_file"]
        if not f.name.lower().endswith(".csv"):
            raise forms.ValidationError("Upload the broker referrals file as
a .csv.")
        if f.size > 5 * 1024 * 1024:
            raise forms.ValidationError("The file is larger than 5 MB.")
        return f
```

- **`fields`** lists only what the user fills in. The counts, `created_by` and `result_file` are set by the view, never by the user.
- **`type="month"`** makes the browser send `2026-08`, exactly the format `month_range` expects.
- **Every `clean_` method must `return` the value.** Forgetting the return saves `None`.
- **Two layers of checks:** the form checks what it can see (format, size, extension); `services.py` checks what needs the data (missing columns).

Step 7: views.py, the four pages

A view is a function that receives a request and returns a response: an HTML page, a redirect or a file. Views stay thin: they check the form, call `services.py`, save the run, and hand back a page.

View	Address	Job
`run_list`	`/commission/`	Past runs, newest first
`run_create`	`/commission/new/`	The form; on submit, build and save the report

View	Address	Job
`run_detail`	`/commission/<id>/`	Headline numbers and the first 15 brokers
`run_download`	`/commission/<id>/download/`	The Excel file

The main one: `run_create`

```
@login_required
def run_create(request):
    if request.method == "POST":
        form = CommissionRunForm(request.POST, request.FILES)        # FILES
holds the upload
        if form.is_valid():
            run = form.save(commit=False)                            # a model
object, not saved yet
            run.created_by = request.user
            upload = form.cleaned_data["broker_file"]
            try:
                brokers = services.read_brokers(upload)
                upload.seek(0)                                       # rewind
so Django can save the file too
                report = services.build_commission(get_ledger(), brokers,
run.month, float(run.rate))
                content = excel.build_workbook(report["brokers"],
report["clients"])
            except ValueError as err:
                form.add_error(None, str(err))                       # show the
message at the top of the form
                return render(request, "commission/run_form.html", {"form":
form})

            run.brokers_count = report["summary"]["brokers"]
            ...
            run.save()                                               # the row
and the uploaded CSV
            run.result_file.save(f"Broker_Commission_{run.month}.xlsx",
ContentFile(content))
            messages.success(request, f"Report for {run.month} is ready.")
            return redirect("commission:run_detail", pk=run.pk)    # post /
redirect / get
```

```
    else:
        last_month = (pd.Timestamp.today().to_period("M") - 1).strftime("%Y-
%m")
        form = CommissionRunForm(initial={"month": last_month, "rate":
Decimal("0.05")})
    return render(request, "commission/run_form.html", {"form": form})
```

The ideas to learn here

- **One URL, two methods.** A GET shows an empty form; a POST processes it. An invalid POST falls through to the last `render`, which shows the same form with its errors.
- **`save(commit=False)`** gives a model object you can finish (here, `created_by` and the counts) before writing it to the database.
- **Post / redirect / get.** After a successful POST, `redirect(...)` sends the browser to the detail page. Refreshing that page doesn't submit the form, or generate the report, a second time.
- **`ContentFile(bytes)`** wraps the Excel bytes so a `FileField` can save them under `MEDIA_ROOT`.
- **`@login_required`** sends anyone not logged in to the login page first. Commission data shouldn't be one link away from anyone on the network.
- **`messages.success`** shows a one-time banner on the next page.

Download and preview

```
@login_required
def run_download(request, pk):
    run = get_object_or_404(CommissionRun, pk=pk)                  # 404 page
instead of a crash
    return FileResponse(run.result_file.open("rb"), as_attachment=True,
                        filename=f"Broker_Commission_{run.month}.xlsx")
```

`run_detail` reads the saved Excel back with `pd.read_excel(..., sheet_name="Brokers", header=1)` and turns the first 15 rows into an HTML table with `to_html`, so the page always matches the file.

Step 8: URLs, templates and login

URLs connect addresses to views

```
# commission/urls.py
app_name = "commission"                    # lets templates write {% url
'commission:run_list' %}
urlpatterns = [
    path("", views.run_list, name="run_list"),
    path("new/", views.run_create, name="run_create"),
    path("<int:pk>/", views.run_detail, name="run_detail"),          #
<int:pk> becomes the pk argument
    path("<int:pk>/download/", views.run_download, name="run_download"),
]

# nexora_reports/urls.py (the project)
urlpatterns = [
    path("", RedirectView.as_view(pattern_name="commission:run_list")),
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")),         #
Django's own login and logout
    path("commission/", include("commission.urls")),
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

Use the `name`, never the address, in templates and redirects: `{% url 'commission:run_detail' run.pk %}`. If an address changes later, every link follows.

Templates: one layout, small pages

`base.html` holds the header, styles and message banner, with an empty `{% block content %}`. Each page **extends** it and fills only that block:

```
{% extends "commission/base.html" %}
{% block content %}
<form method="post" enctype="multipart/form-data">
  {% csrf_token %}
  {{ form.non_field_errors }}
  {{ form.as_p }}
  <button class="btn" type="submit">Generate report</button>
</form>
{% endblock %}
```

- **`enctype="multipart/form-data"`** is required for file uploads. Without it, `request.FILES` is empty and the form says the file is missing.
- **`{% csrf_token %}`** protects every POST form from forged submissions; Django refuses a POST without it.
- **`{{ form.as_p }}`** draws each field with its label and errors.
- **`{% for run in runs %} ... {% empty %} ... {% endfor %}`** in the list page shows a message when there are no runs yet.

Login

`include("django.contrib.auth.urls")` provides the login and logout pages. All it needs is a template at `templates/registration/login.html`. Logout is a small POST form in the header, as current Django versions require.

Step 9: tests

`python manage.py test commission` runs nine tests in about six seconds, on a temporary database that's thrown away afterwards. Run them after every change.

Testing the calculation on a table you can check by hand

```
class ServiceTests(TestCase):
    def setUp(self):
        self.led = pd.DataFrame({
            "ClientID":   ["A", "A", "A", "B", "B"],
            "CreatedAt":  pd.to_datetime(["2026-08-02", "2026-08-03", "2026-
08-04", "2026-08-05", "2026-09-01"]),
            "LedgerType": ["BILL", "bill ", "NEGATIVE BALANCE PROTECTION",
"BILL", "BILL"],
            "Debit":      [100.0, None, 0.0, 0.0, 500.0],
            "Credit":     [0.0, 30.0, 10.0, 80.0, 0.0],
        })

    def test_client_pnl(self):
        pnl = services.client_pnl(self.led, ["A", "B"],
*services.month_range("2026-08")).set_index("ClientID")
        self.assertEqual(pnl.loc["A", "Net_Profit"], -60)          # 30 - 100
+ 10
        self.assertAlmostEqual(pnl.loc["A", "Commission"], 3.0)    # 5% of 60
        self.assertEqual(pnl.loc["B", "Net_Profit"], 80)           # the
September loss is outside August
```

The tiny table deliberately includes the messy cases: `'bill '` with a space, a blank Debit, an NBP row, and a row in the wrong month. Each check is a number you can work out on paper.

Testing the pages like a user

```
def test_create_report(self):
    self.client.login(username="analyst", password="pass12345")
    upload = SimpleUploadedFile("brokers.csv", SAMPLE_CSV.read_bytes(),
content_type="text/csv")
    response = self.client.post(reverse("commission:run_create"),
                                {"month": "2026-08", "rate": "0.05",
"broker_file": upload})
    run = CommissionRun.objects.get()
    self.assertRedirects(response, reverse("commission:run_detail", args=
[run.pk]))
    self.assertEqual(run.brokers_count, 5)
```

- **`self.client`** is a fake browser: log in, post a form, follow a link.

- **`override_settings(MEDIA_ROOT=temp_folder, LEDGER_LOADER=demo)`** keeps tests away from your real files and real ledger.
- **The other tests cover** the broker sheet rules, a missing ledger column, the login requirement, a badly formatted month being rejected, and the ledger slice: only the asked-for month, clients and types come back; a loader without `scan_ledger()` gives the same rows; and unreadable dates are reported.

Step 10: run it, then switch to your ledger

First with the demo data

```
cd nexora_reports
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open `http://127.0.0.1:8000`, log in, click **New report**, choose `commission/sample_data/brokers_sample.csv`, month `2026-08`, and **Generate report**. You should land on a page showing 5 brokers and 30 clients, with a **Download Excel** button and a preview of the Brokers sheet.

Then with your real ledger

1. Replace your `ledger_loader.py` with the updated one. It's your file with `scan_ledger()` added; `get_final_ledger()` returns exactly what it did before, so your notebooks keep working.
2. Install the requirements once: `pip install -r requirements.txt` (adds `polars` and `pyarrow` if missing).
3. Near the bottom of `nexora_reports/settings.py`:

```
LEDGER_MODULE = "ledger_loader"
LEDGER_MODULE_PATH = r"C:\Users\Administrator\Videos\Office\migration"    #
the folder holding ledger_loader.py
LEDGER_ONLY_NEW_APPENDS = False       # True if append files can overlap
Ledger.csv
```

Restart the server and upload your real `brokers-with-referrals-*.csv`. Each report reads its month straight from `Ledger.csv` and the append folder, using the paths inside `ledger_loader.py`. An amber banner appears if any rows have an unreadable

`CreatedAt`, and another if the month has no BILL or NBP rows for those clients (usually a wrong month or path).

Check it against the notebook

Run the same month in both, then compare the Brokers sheets:

```
app = pd.read_excel(r"...\Broker_Commission_2026-08.xlsx",
sheet_name="Brokers", header=1)
nb = pd.read_excel(r"...\Broker_Commission_Aug.xlsx", sheet_name="Brokers",
header=1)
merged = app.merge(nb, on="Parent", suffixes=("_app", "_nb"))
merged[merged["Total Loss(Debit)_app"].round(2) != merged["Total
Loss(Debit)_nb"].round(2)]
```

Profit and loss should match. NBP, and anything that depends on it, will differ where the notebook dropped NBP rows: that difference is the fix.

Next steps, in the order they'd matter

1. Make each report faster with Parquet

Each report now reads only the rows it needs, but a CSV still has to be scanned from top to bottom to find them. Converting the ledger to Parquet once a day lets Polars skip whole blocks of rows that fall outside the month:

```
from ledger_loader import scan_ledger
scan_ledger(columns=None).sink_parquet(r"C:\...\ledger.parquet")      #
nightly: one streaming pass

# in a scan_ledger_parquet() for the app:
pl.scan_parquet(r"C:\...\ledger.parquet").filter(...).collect()         #
reads only the needed blocks
```

Point `LEDGER_MODULE` at a module whose `scan_ledger()` reads the Parquet file, and nothing else in the app changes.

2. Read the broker list from PostgreSQL instead of uploading it

The brokers-with-referrals CSV comes out of the CRM. Django can query `centric-crm` directly by adding it as a second database in `DATABASES` and running the same SQL you'd use in Metabase with `pd.read_sql`. The form then needs only the month and the rate.

3. Run long reports in the background

If a report takes more than about 30 seconds, the browser may time out. A task queue runs it outside the request while the page shows "processing". For one Windows server, `django-q2` is simpler than Celery. The `CommissionRun` model would gain a `status` field: pending, done or failed.

4. Deploy for the team

- **`DEBUG = False`**, a real `SECRET_KEY` from an environment variable, and `ALLOWED_HOSTS` set to the server's name.
- **Serve it with Waitress** on Windows (`pip install waitress`, then `waitress-serve nexora_reports.wsgi:application`), not `runserver`.
- **Switch to PostgreSQL** in `DATABASES` instead of the default SQLite file.
- **Serve `/media/`** through the web server, because the `static(...)` line only works in `DEBUG`.

5. Limit who sees what

`@login_required` lets any logged-in user see every run. Django's permissions (`@permission_required("commission.view_commissionrun")`) or a "Finance" group can limit it to the people who should see commission figures.

Practice: build these yourself

Each one touches a different part of Django. Do them in order, run `python manage.py test commission` after each, and add a test for the feature.

- [ ] **Notes on a run.** Add `notes = models.TextField(blank=True)` to the model, `makemigrations` + `migrate`, add it to the form's `fields`, and show it on the detail page. *Teaches: migrations, forms, templates.*

- [ ] **Filter the list by month.** Read `request.GET.get("month")` in `run_list` and filter `CommissionRun.objects.filter(month=...)`; add a small GET form above the table. *Teaches: query parameters, QuerySets.*
- [ ] **Delete a run.** A POST-only view that deletes the row **and** both files (`run.result_file.delete()`), with a confirm page. *Teaches: POST-only actions, files.*
- [ ] **Rate from settings.** Move the default 0.05 into `settings.DEFAULT_COMMISSION_RATE` and use it in the form's `initial`. *Teaches: settings as configuration.*
- [ ] **A "Clients" preview tab.** Show the first 15 rows of the Clients sheet below the Brokers preview. *Teaches: reusing a view pattern.*
- [ ] **Compare two months.** A page that takes two runs and shows, per broker, the brokerage in each and the change. *Teaches: combining services.py functions with a new view; the pandas part is a merge you already know.*
- [ ] **A ledger health page.** A staff-only page showing the latest `CreatedAt` in `Ledger.csv` and in the append files, and the unreadable-date count, from `scan_ledger()` with one `pl.collect_all`. *Teaches: `@user_passes_test`, reusing the loader.*